

Sprint report

Sprint	Sprint 2
Period	2nd and 3rd week
Contributor	Oussama Aaraba
Repository / tag	sprint-2

1. Summary

Sprint objective: replace the dummy model with a genuinely trained detection model, and integrate it into the existing application.

The full chain now runs on real detections: the user uploads blade photos, the model identifies the damage, severity is computed and the areas are displayed on the images. The model reaches 0.805 mAP50 on the test set, and 0.953 for the crack class, the most critical one operationally.

Two points remain open: the corrosion class has a low recall (0.580), and a single elongated defect can produce two overlapping detections. Neither prevents the tool from being used.

2. Model

Dataset

The model is trained on WTBD (Ji, Cheng & Wu, Scientific Data 13:396, 2026), a public set of 1,065 drone-captured blade images annotated into six defect categories: craze, corrosion, surface_injure, thunderstrike, crack and hide_craze.

Split: 745 training images, 159 validation, 161 test. The dataset is deliberately difficult — the authors report high intra-class variability and a marked visual similarity between some categories.

The dataset was re-hosted on Hugging Face and the Pascal VOC annotations converted to YOLO format (coordinates normalised between 0 and 1).

Method

Fine-tuning of YOLOv8s pre-trained on COCO. The backbone and neck of the network keep the weights learned on COCO; only the detection heads are rebuilt for six classes instead of eighty, then the whole network is fine-tuned on the blade images.

Input resolution raised to 1024 pixels rather than 640: cracks are thin structures that disappear at low resolution. Random seed fixed at 42 to guarantee reproducibility across runs.

Comparison of the two training runs

A first run of 50 epochs served as a baseline. The curves showed that validation losses and mAP were still improving at the last epoch, with no sign of overfitting: the model had not converged. A second run of 100 epochs was therefore launched from the same COCO weights.

Restarting from scratch rather than extending the first model is a deliberate choice: the learning rate follows a decreasing schedule sized on the total number of epochs. Resuming an already converged model would have re-applied a high rate to stabilised weights.

Metric	50 epochs	100 epochs
mAP50 (validation)	0.785	0.823
mAP50-95 (validation)	0.475	0.522
Mean recall	0.747	0.778
crack — mAP50	0.967	0.993
False positives on clean background	74	55

Duration	0.50 h	1.08 h
----------	--------	--------

The second run improves every metric. The gain in mAP50-95 (0.475 to 0.522) is the most significant one: it reflects better fitted boxes, not simply more objects found. This matters because severity is computed from the box area.

The drop in false positives on clean background — from 74 to 55, i.e. 26 % — addresses the main weakness identified in the first run.

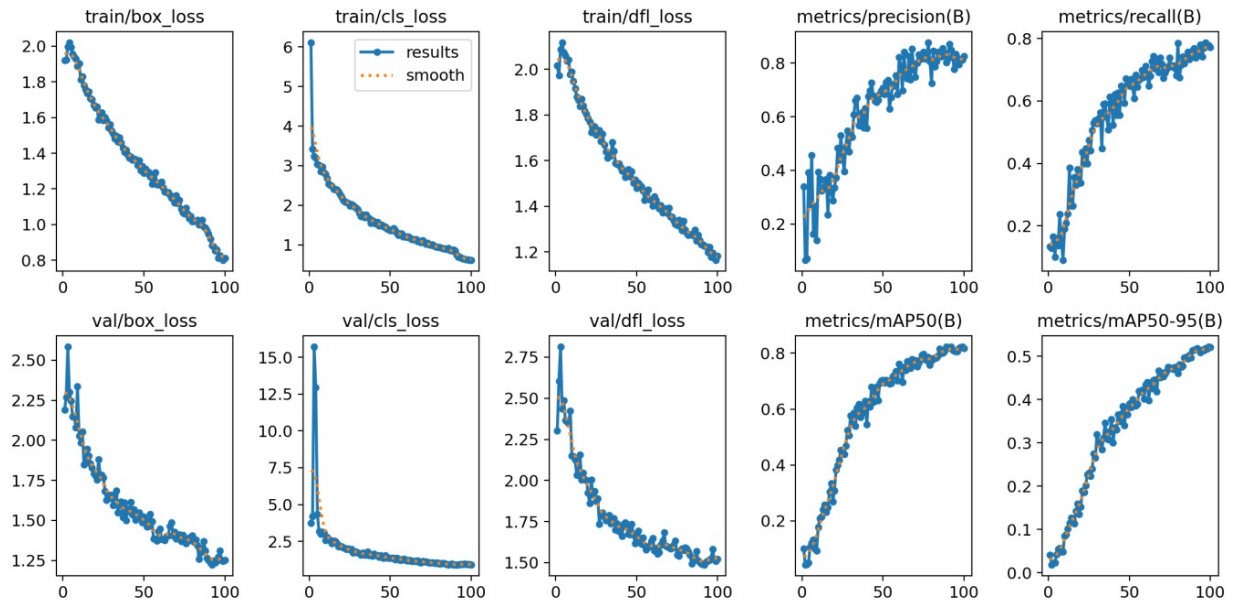


Figure 1 — Training curves over 100 epochs. Validation losses decrease without rebound: no overfitting. The plateau reached around epoch 85 indicates convergence.

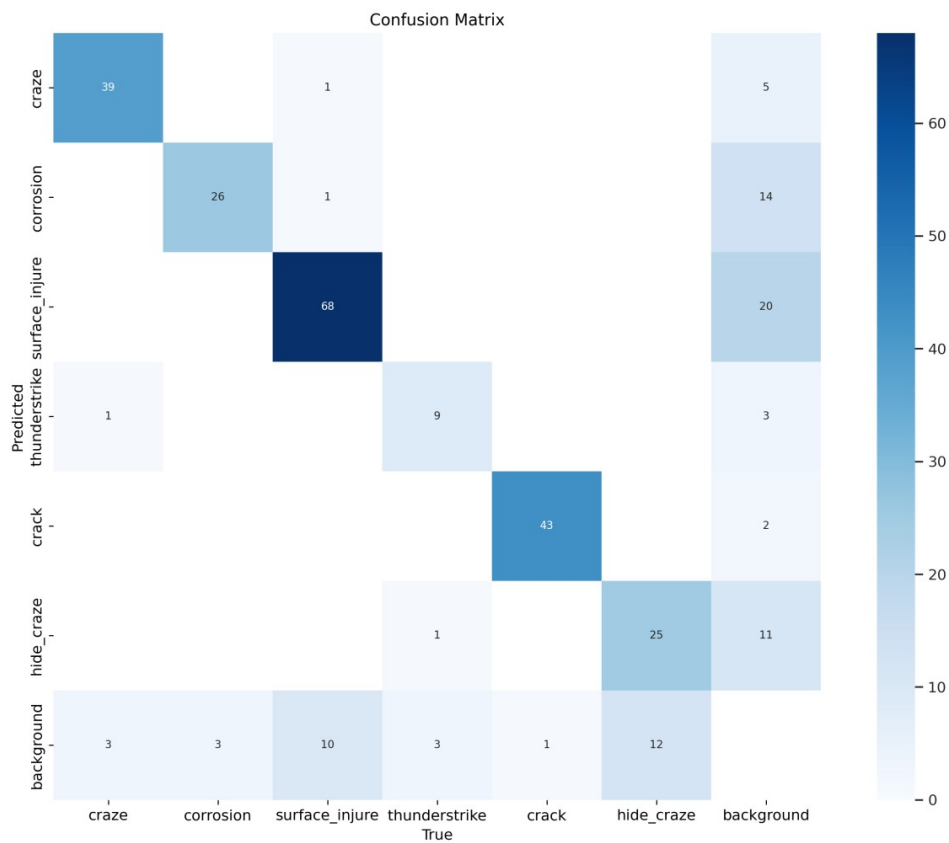


Figure 2 — Confusion matrix, raw counts (validation set).

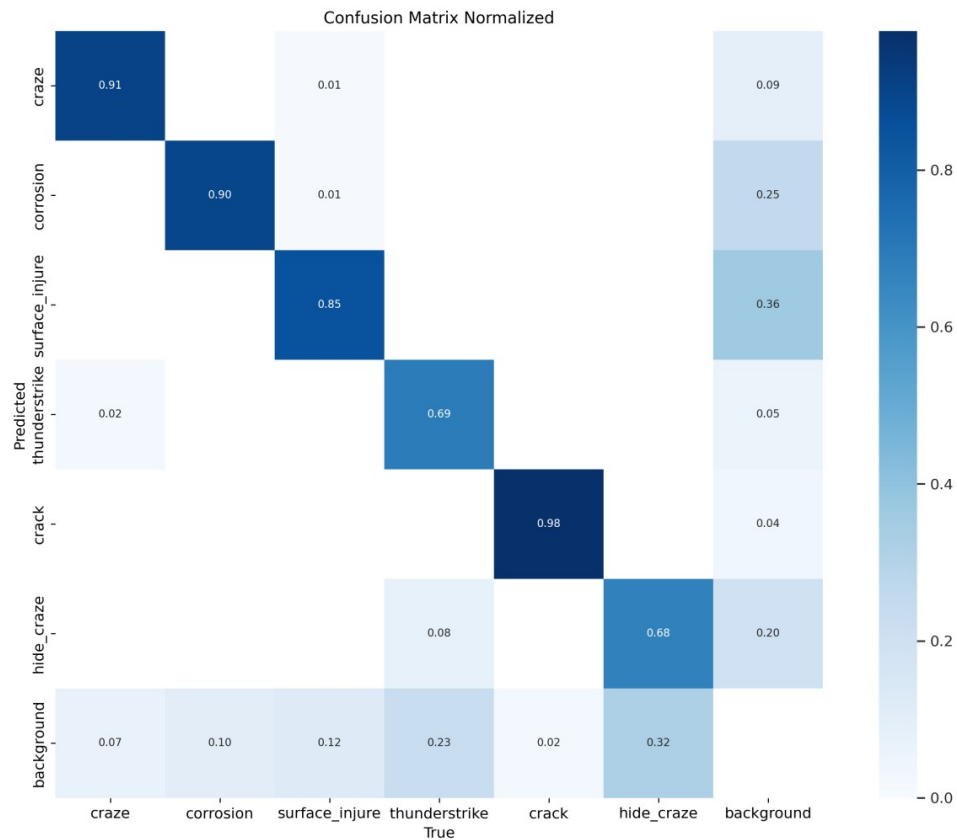


Figure 3 — Normalised confusion matrix. The diagonal carries the per-class detection rates.

The matrices reveal an important point: the classes are almost never confused with one another. Off-diagonal cells between categories are worth 0.01 or 0.02. The model's difficulty is not about the nature of the damage but about its presence — the background column concentrates most of the errors.

This has a practical consequence: merging classes judged to be close, such as craze and hide_craze, would bring nothing. The effort must go into reducing false alarms.

Test set results

The metrics below are measured on the test set, never used during training. Validation values are deliberately set aside: the retained weights were selected on that set, so their metrics are optimistic.

Class	Precision	Recall	mAP50	mAP50-95
thunderstrike	1.000	0.959	0.995	0.661
crack	0.933	0.839	0.953	0.619
surface_injure	0.835	0.766	0.786	0.561
hide_craze	0.871	0.679	0.728	0.448
craze	0.725	0.711	0.709	0.414
corrosion	0.748	0.580	0.661	0.317
All classes	0.852	0.756	0.805	0.503

The gap between validation (0.823) and test (0.805) is small, which confirms that the model generalises.

Reading the results

- crack reaches 0.953 mAP50 with 0.839 recall. It is the most critical class: a crack propagates under cyclic load and leads to failure. It is also the best detected.
- thunderstrike shows 0.995, but on only 17 instances. The figure is not statistically reliable and should not be presented as a solid result.

- corrosion has the lowest recall (0.580): nearly half of the corroded areas escape the model. Corrosion is diffuse, with no clear boundary, which also makes human annotation harder.
- hide_craze remains difficult (0.728): by definition, barely visible crazing offers little signal.

Publication

The weights are hosted on Hugging Face and pinned by version tag. The backend downloads a specific revision, never the main branch: a retraining therefore cannot silently change the API's behaviour. The version used is recorded with every inspection.

- v1 — 50-epoch run, kept as a fallback.
- v2 — 100-epoch run, currently deployed version.

3. Backend

What changed

No change to the architecture. The contract fixed in the previous sprint held: only the body of the prediction function was rewritten, and the services, routes and schemas stayed unchanged.

Changes made

- Damage class enumeration aligned with the WTBD dataset: the four provisional values (crack, erosion, lightning_strike, delamination) are replaced by the six real categories.
- Alembic migration of the PostgreSQL ENUM type. Adding and removing values at the same time requires recreating the type and reattaching the column. Detections produced by the dummy model were deleted: their classes no longer existed.
- Inference service rewritten: weights loaded from Hugging Face, cached for the lifetime of the process, model outputs converted to the expected format.
- Model preloaded at API startup rather than on the first request.
- Severity rules adapted to the six classes, weighted by structural seriousness: a crack outranks a surface defect of equal extent.
- Confidence threshold raised to 0.40 instead of the 0.25 default, in response to the false positives seen in the confusion matrix.

Technical decisions

Each class label is read from the weights file itself, never from a hard-coded table. A retraining that reordered the classes would otherwise rename every detection without raising the slightest error.

Box coordinates are clamped to the $[0, 1]$ interval after conversion. A box touching the image edge can exceed it by a thousandth after rounding, which the database constraints would reject.

The 0.40 confidence threshold reflects an accepted trade-off: the tool is used for triage before human review. Drowning an operator in false alarms costs more than missing a minor defect.

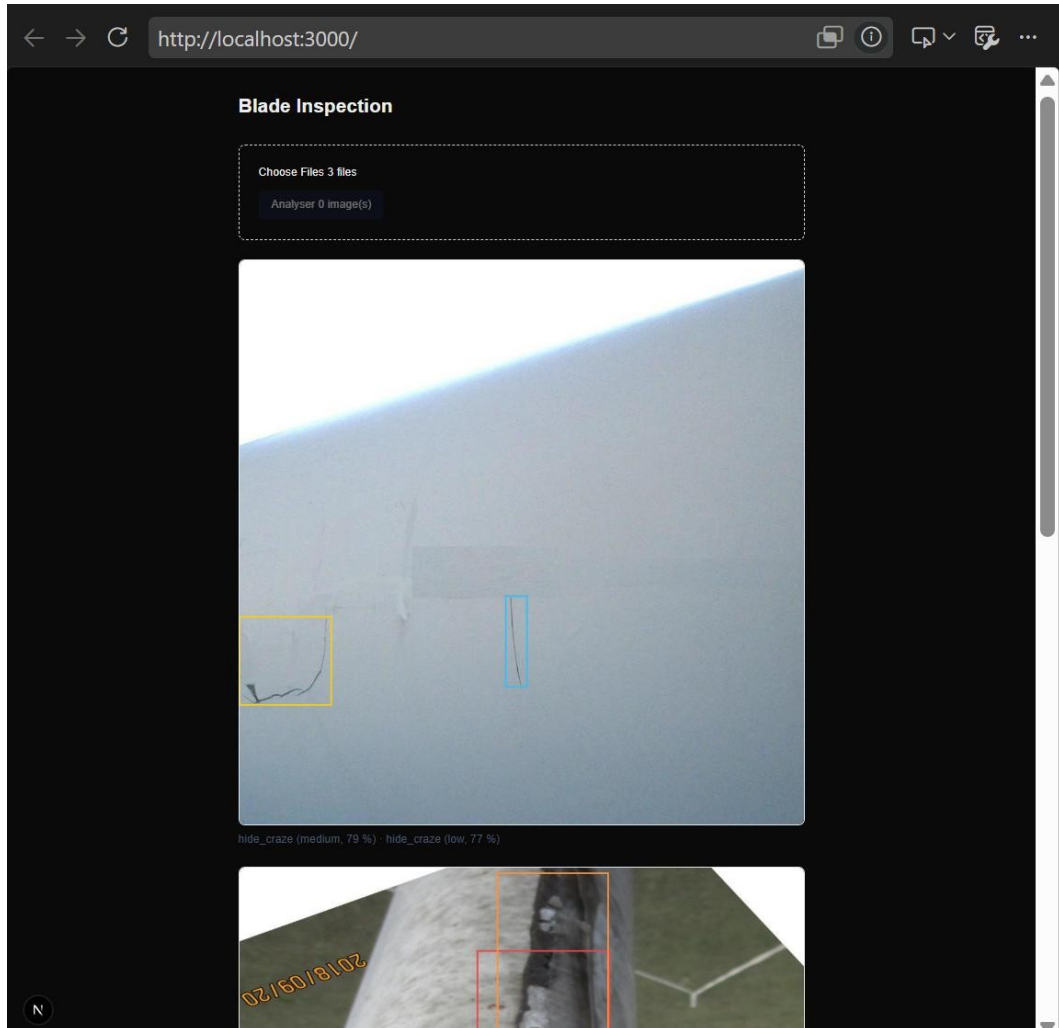
Known limitations

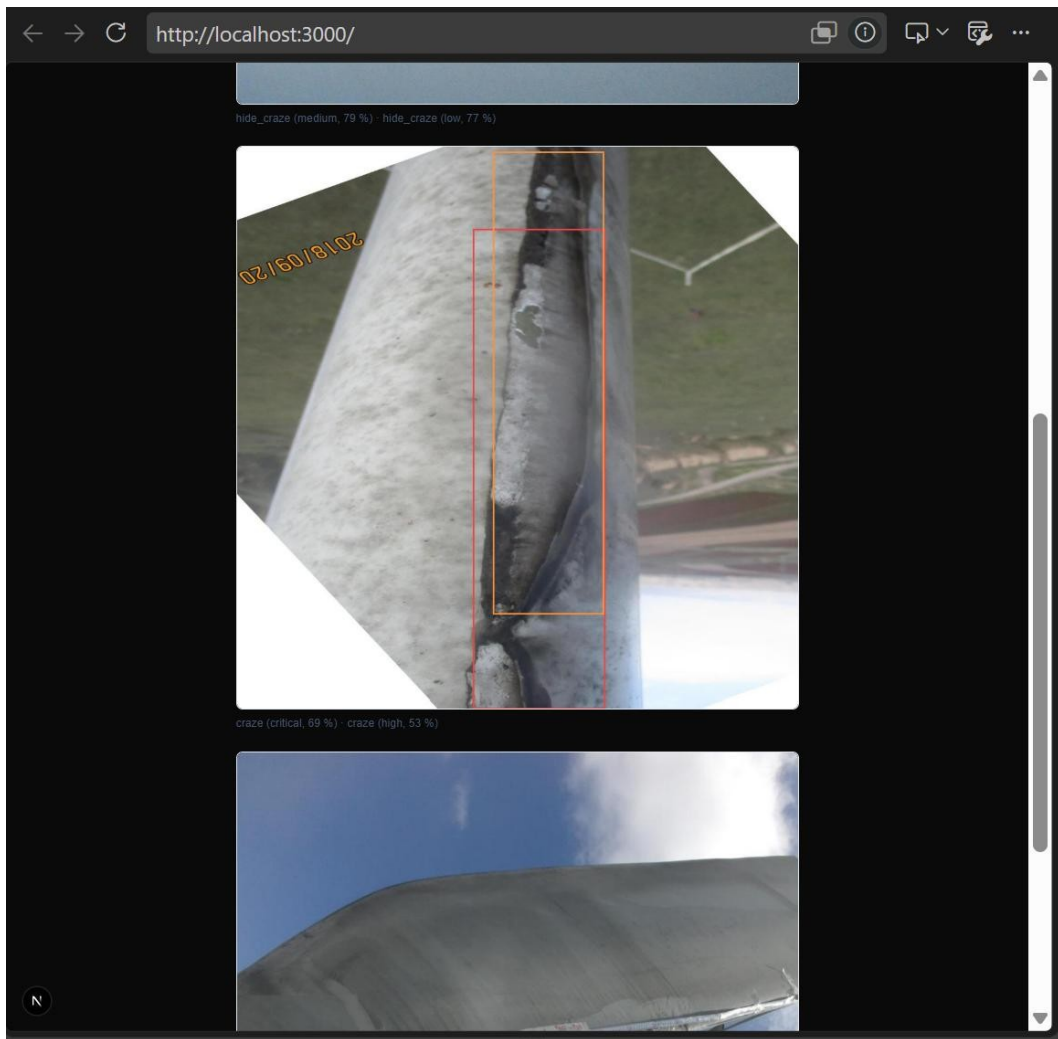
- The relative area used for the severity computation measures the share of the image occupied, not a physical area. Two photos taken at different distances are not directly comparable.
- An elongated defect can produce two overlapping detections: duplicate suppression does not always merge boxes of the same class spread over several grid cells.
- Inference runs inside the API process. Acceptable at this scale, to be moved to a task queue for production use.

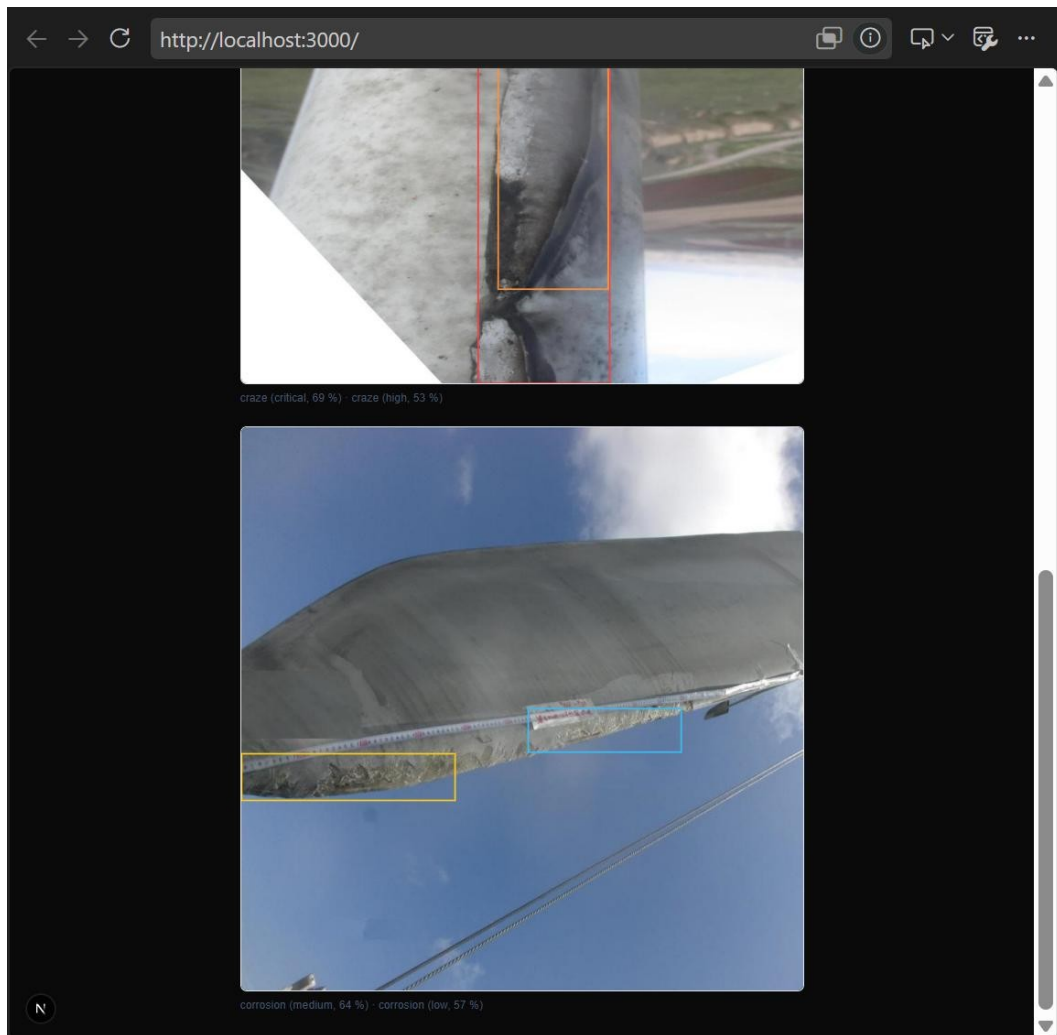
4. Frontend

No change during this sprint. The interface built in sprint 1 displays the real detections with no code change, the exchange contract having stayed identical.

The screenshots below show detections produced by the v2 model on blade images.







Remaining limitation: the turbine identifier is still hard-coded in the interface. Its selection is planned for the next sprint.

5. Next sprint

- Add the turbine list and its selection in the interface, replacing the hard-coded identifier.
- Merge overlapping detections of the same class, or adjust the overlap threshold.
- Build the per-turbine report view: damage summary, breakdown by severity, export.
- Handle the upload of large batches: the current limit is fifty images per inspection.
- Assess the value of healthy blade images as negative examples, to reduce false alarms.

Blockers and open questions

- Are the six WTBD categories suitable, or do you have a damage nomenclature used by your clients?
- Do you have internally annotated blade images? The public dataset is small, and that is the main limit to the model's progress.
- Which shared repository should be used for sprint deliveries?
- The current confidence threshold favours precision over recall. Does this trade-off match the intended use?